



Modern JS & Web API'ler

ES6 ile Daha Temiz Kod, Web API'ler ile Daha Güçlü Uygulamalar

JS

ES6 Öncesi ve Sonrası – Değişim

Değişken Tanımlama (var vs. let ve const)

ES5

```
function example() {  
  if (true) {  
    var x = 10;  
  }  
  console.log(x); // Çıktı: 10 (Block scope yok)  
}  
example();
```

var, fonksiyon scope'unda çalışır ve değişken tanımlamaları hoisting nedeniyle yukarı taşınır.

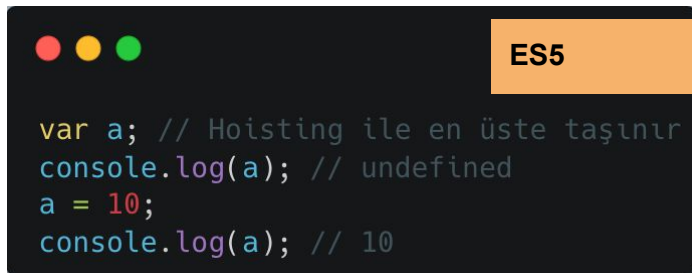
ES6

```
function example() {  
  if (true) {  
    let x = 10;  
    const y = 20;  
  }  
  console.log(x); // Hata: x is not defined  
}  
example();
```

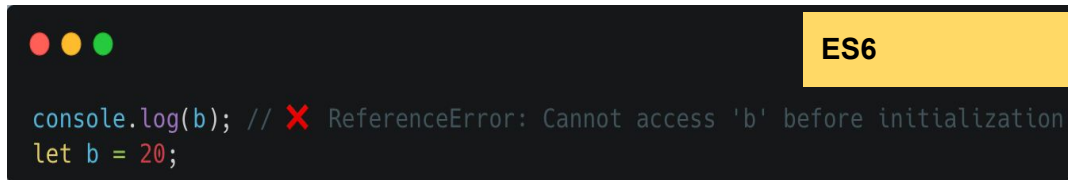
let ve const, block scope'a sahiptir. const sabit değişkenler için kullanılır.

ES6 Öncesi ve Sonrası – Değişim

hoisting kavramı JavaScript'in değişken ve fonksiyon tanımlarını çalıştırılmadan önce belleğe “taşması” anlamına gelir.

A code editor window with a dark background and three colored window control buttons (red, yellow, green) in the top-left corner. The title bar on the right is orange and labeled "ES5". The code is written in a light blue monospace font.

```
var a; // Hoisting ile en üste taşınır
console.log(a); // undefined
a = 10;
console.log(a); // 10
```

A code editor window with a dark background and three colored window control buttons (red, yellow, green) in the top-left corner. The title bar on the right is yellow and labeled "ES6". The code is written in a light blue monospace font.

```
console.log(b); // ❌ ReferenceError: Cannot access 'b' before initialization
let b = 20;
```

Hataları yakalamamıza yardımcı olur. Bir değişkene tanımlanmadan önce erişmeye çalışmak yanlıştır ve mümkün olmamalıdır

*let ve const ile tanımlanan değişkenler temporal dead zone (TDZ) içinde kalır ve tanımlanmadan önce kullanıldığında hata fırlatır.

ES6 Öncesi ve Sonrası – Değişim

Template Literals (+ yerine `` kullanımı)

ES5

```
var name = "Emre";  
var message = "Merhaba, " + name + "! Hoş geldin.";  
console.log(message);
```

ES6

```
let name = "Emre";  
let message = `Merhaba, ${name}! Hoş geldin.`;  
console.log(message);
```

ES6 Öncesi ve Sonrası – Değişim

Destructuring (Nesne ve Dizileri Parçalama)

ES5

```
var person = { name: "Emre", age: 26 };  
var name = person.name;  
var age = person.age;  
console.log(name, age);
```

ES6

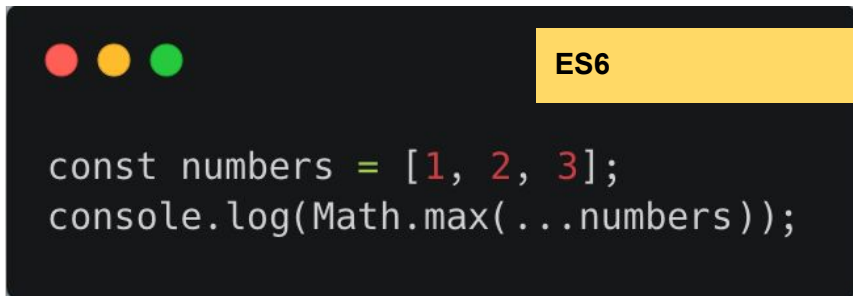
```
const person = { name: "Emre", age: 26 };  
const { name, age } = person;  
console.log(name, age);
```

ES6 Öncesi ve Sonrası – Değişim

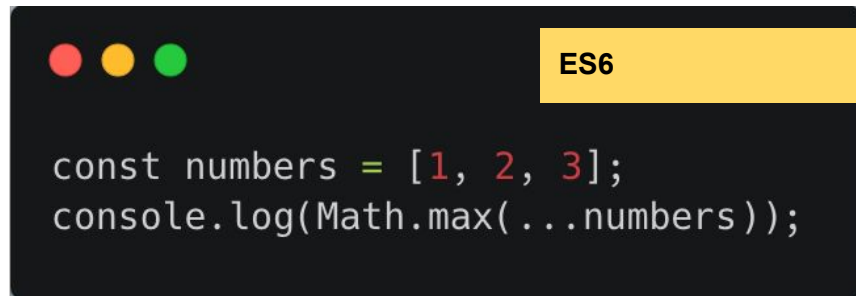
Spread ve Rest Operator (...)

A code editor window with a dark background and three colored window control buttons (red, yellow, green) in the top-left corner. The title bar is orange and contains the text "ES5".

```
var numbers = [1, 2, 3];  
console.log(Math.max.apply(null, numbers));
```

A code editor window with a dark background and three colored window control buttons (red, yellow, green) in the top-left corner. The title bar is yellow and contains the text "ES6".

```
const numbers = [1, 2, 3];  
console.log(Math.max(...numbers));
```

A code editor window with a dark background and three colored window control buttons (red, yellow, green) in the top-left corner. The title bar is yellow and contains the text "ES6".

```
const numbers = [1, 2, 3];  
console.log(Math.max(...numbers));
```

ES6 Öncesi ve Sonrası – Değişim

Arrow Functions (function yerine =>)

ES6

```
const multiply = (a, b) => a * b;  
console.log(multiply(5, 2));
```

ES5

```
var multiply = function(a, b) {  
    return a * b;  
};  
console.log(multiply(5, 2));
```

ES6 Öncesi ve Sonrası – Değişim

Classes (ES6 Class Kullanımı)

Öncesi (ES5 – Prototip Kullanımı)

ES5

```
function Person(name, age) {  
  this.name = name;  
  this.age = age;  
}  
Person.prototype.sayHello = function() {  
  console.log("Merhaba, benim adım " + this.name);  
};  
var emre = new Person("Emre", 26);  
emre.sayHello();
```

Sonrası (ES6 – Class Kullanımı)

ES6

```
class Person {  
  constructor(name, age) {  
    this.name = name;  
    this.age = age;  
  }  
  sayHello() {  
    console.log(`Merhaba, benim adım ${this.name}`);  
  }  
}  
const emre = new Person("Emre", 26);  
emre.sayHello();
```


ES6 Öncesi ve Sonrası – Değişim

Default Parameters (Varsayılan Parametreler)

Öncesi (ES5 – Manuel Kontrol)

ES5

```
function greet(name) {  
    name = name || "Misafir";  
    console.log("Merhaba, " + name);  
}  
greet();
```

Sonrası (ES6 – Default Parameter Kullanımı)

ES6

```
const greet = (name = "Misafir") =>  
    console.log(`Merhaba, ${name}`);  
greet();
```

ES6 Öncesi ve Sonrası – Değişim

Modules (import/export)

Öncesi (ES5 – IIFE veya CommonJS)

```
ES5

// math.js (Modül)
function add(a, b) {
  return a + b;
}

// Modülü dışa aktarma
module.exports = add;

// main.js (Ana Dosya)
const add = require("./math.js");
console.log(add(2, 3)); // 5
```

Bu yöntem sadece Node.js ortamında çalışıyordu, tarayıcıda desteklenmiyordu.

Sonrası (ES6 – Modules)

```
ES6

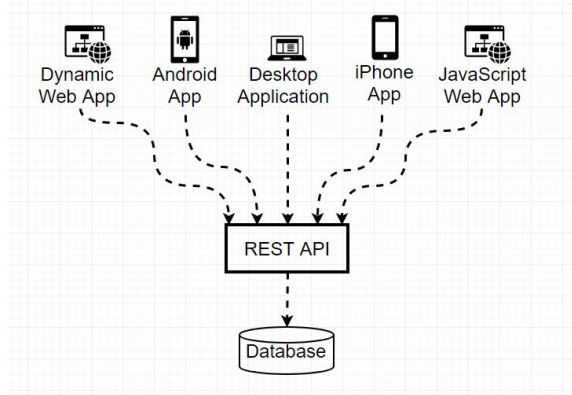
// module.js
export const sayHello = () =>
  console.log("Merhaba Dünya");

// main.js
import { sayHello } from "./module.js";
sayHello();
```

Web API'ler Nedir?

Tarayıcının sağladığı fonksiyonlar (localStorage, Fetch, DOM API vs.)

Back-end çağrıları ve kullanıcı bilgileri ile çalışma

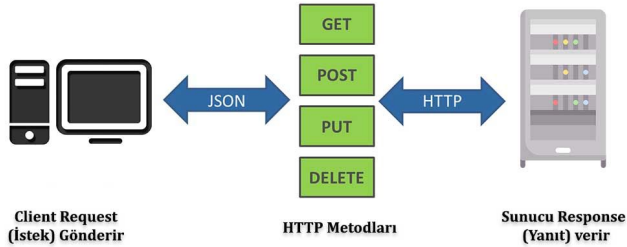


Google Maps API, hava durumu API'leri, ödeme sistemleri, veri çekme (Fetch API)



Web API'ler Nedir?

Rest API Çalışma Metodu



- **GET**, veri listelemek ve görüntülemek için kullanılır.
- **POST**, veri eklemek için kullanılan bir istektir.
- **DELETE**, veri silmek için kullanılır.
- **PATCH**, verinin bir kısmının güncellenmesi için kullanılan bir istektir.

Web API'ler Nedir?

XMLHttpRequest

```
const xhr = new XMLHttpRequest();
xhr.open('GET', 'https://jsonplaceholder.typicode.com/posts/1');
xhr.send();
xhr.onload = () => {
  console.log(xhr.responseText);
}
```

Basit Fetch API Kullanımı:

```
fetch("https://jsonplaceholder.typicode.com/todos/1")
  .then(response => response.json())
  .then(data => console.log(data));
```

Web API'ler Nedir?

localStorage & sessionStorage

localStorage → Kalıcıdır (Tarayıcı kapanınca bile saklanır). 5MB'a kadar veri tutulur

SET, GET, REMOVE, CLEAR methodlarımız var.

sessionStorage → Geçicidir (Sayfa kapanınca silinir).



```
localStorage.setItem("username", "Emre");  
console.log(localStorage.getItem("username"));
```

Web API'ler Nedir?

Sık kullanılanlar

- Clipboard API – Kullanıcının Panosuna Veri Kopyalama
- Notification API – Tarayıcı Bildirimleri Gönderme
- Device Orientation API – Cihazın Hareket Sensörlerini Kullanma

Bonus

- Battery API – Cihazın Pil Durumunu Kontrol Etme
- Speech Recognition API – Kullanıcı Sesini Tanıma

Bazı Kaynaklar

<https://medium.com/sliit-foss/es5-vs-es6-in-javascript->

<https://ugurgeliskan.medium.com/javascript-yap%C4%B1c%C4%B1-fonksiyonlar-constructor-54952d5fd7c4#:~:text=Terim%20olarak%20ismi%20ile%20Generator,tekrar%20tekrar%20girilip%20%C3%A7%C4%B1k%C4%B1labilen%20fonksiyonlard%C4%B1r>

<https://www.hosting.com.tr/bilgi-bankasi/rest-api/>

<https://jsonplaceholder.typicode.com/>

<https://developer.mozilla.org/en-US/docs/Web/API/XMLHttpRequest/readyState>